

Introdução ao MongoDB com PyMongo

Banco de Dados

Prof. Dr. Carlos Melo

 carlos.melo@upe.br



O que é NoSQL?

- Bancos Not Only SQL (NoSQL):
 - Não dependem exclusivamente de tabelas relacionais
 - oferecem modelos de dados mais flexíveis
 - podem trabalhar com documentos, grafos, chave-valor, colunas, etc.

Tipo	Exemplo
Documento	MongoDB
Chave-valor	Redis
Grafo	Neo4j
Colunar	Cassandra



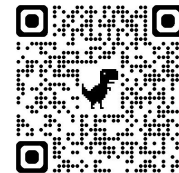
Vamos de Documentos

- O MongoDB é um banco NoSQL orientado a documentos
- Um documento armazena dados em formato JSON/BSON
- Podem possuir listas e objetos internos, além de ser flexível

Relacional	MongoDB
Tabela	Coleção
Linha	Documento
Coluna	Campo

```
1 {  
2   "titulo": "Dark",  
3   "ano": 2017,  
4   "temporadas": 3  
5 }
```

- Cada documento no banco pode ter estruturas diferentes, poderíamos ter uma série com campos a mais ou a menos



Documentos e Coleções

- Uma coleção agrupa documentos relacionados entre si: series
- Documentos são cada "registro"

```
1 {  
2   "titulo": "Dark",  
3   "ano": 2017  
4 }
```

```
1 {  
2   "titulo": "Stranger Things",  
3   "temporadas": 5  
4 }
```



BSON vs JSON

- BSON é uma versão Binária do JSON:
- Mais eficiente, rápida para processamento e com suporte a tipos extras

```
1 {  
2   "usuario": "Carlos",  
3   "idade": 18,  
4   "ativo": true  
5 }
```

- Tipos suportados:
 - string, número, boolean, lista, objeto, data...
- Na prática trabalhamos visualmente como se fosse JSON, mas o que é armazenado é BSON



PyMongo

- É a biblioteca/driver Python para acessar o MongoDB, dentro de um ambiente virtual:
 - `pip install pymongo`
- O PyMongo permite:
 - Conectar ao banco
 - Inserir documentos
 - Consultar dados
 - Atualizar documentos
 - Remover documentos





Conectando ao MongoDB

- O MongoClient cria a conexão com o MongoDB que está sendo executado localmente na porta padrão 27017



```
1 from pymongo import MongoClient
2
3 client = MongoClient("mongodb://localhost:27017/")
```



Selecionando um banco de dados e coleção

- streamflix é o nome do banco de dados
- series é a coleção



```
1 db = client["streamflix"]  
2  
3 colecao = db["series"]
```

- bancos e coleções podem ser criados "automaticamente", mesmo que ainda não existam.



Populando a coleção

- `insert_one()` é o método usado para inserir um documento à coleção

```
1 serie = {  
2     "titulo": "Dark",  
3     "ano": 2017,  
4     "temporadas": 3  
5 }  
6  
7 colecao.insert_one(serie)
```

- O MongoDB adiciona automaticamente um `_id` ao documento inserido.
- Também existe um `insert_many()`

```
1 dados = [  
2     {"nome": "Monitor", "preco": 1200},  
3     {"nome": "Teclado", "preco": 150},  
4     {"nome": "Mouse", "preco": 80}  
5 ]  
6  
7 resultado = colecao.insert_many(dados)
```



Consultando dados

- O método `find()` retorna os documentos da coleção
 - (ObjectId é um tipo especial BSON)



```
1 for serie in colecao.find():  
2     print(serie)
```



```
1 {  
2     '_id': ObjectId(...),  
3     'titulo': 'Dark',  
4     'ano': 2017,  
5     'temporadas': 3  
6 }
```

- Também é possível aplicar filtros ao `find()`, como:



```
1 colecao.find({"titulo": "Dark"})
```



Consultando dados

- Também é possível usar os operadores em nossas consultas

Operador	Significado
\$gt	maior que
\$lt	menor que
\$gte	maior ou igual
\$lte	menor ou igual
\$ne	diferente

```
1 colecao.find({  
2   "temporadas": {"$gte": 3}  
3 })
```



Consultando dados

- Além disso podemos utilizar regex (expressões regulares) para encontrar algo

```
1 colecao.find({  
2   "titulo": {  
3     "$regex": "^d",  
4     "$options": "i"  
5   }  
6 })
```

- O options: i implica em uma busca do tipo case insensitive e o ^d indica que o titulo deve começar com a letra d



Atualizando dados

- O `update_one()` permite atualizar um documento, que será filtrado no primeiro argumento, o segundo já é utilizado para determinar o quê será atualizado via `$set`

```
1 colecao.update_one(  
2     {"titulo": "Dark"},  
3     {"$set": {"temporadas": 4}}  
4 )
```

- Também existe um `update_many()`



Removendo dados

- O `delete_one()` permite remover um documento com base na filtragem, do mesmo modo o `delete_many()`

```
1 colecao.delete_many({  
2     "ano": {"$lt": 2010}  
3 })
```

- Sem filtros todos os documentos poderão ser removidos <cuidado!>



Documentos embutidos

- O Mongo permite que a inserção de objetos dentro de objetos (dicionários dentro de dicionários)

```
1 {  
2   "titulo": "Dark",  
3   "ano": 2017,  
4   "diretor": {  
5     "nome": "Baran bo Odar",  
6     "pais": "Alemanha"  
7   }  
8 }
```

- No SQL geralmente faríamos um JOIN de duas tabelas, mas o Mongo frequentemente armazenada dados relacionados juntos, reduzindo a necessidade de algo do tipo, mas ainda assim se precisássemos de um JOIN poderíamos usar um **\$lookup**



Listas no MongoDB

- Tal qual em qualquer JSON é comum termos listas associadas a campos

```
1 {  
2   "titulo": "Dark",  
3   "generos": [  
4     "Drama",  
5     "Ficção",  
6     "Suspense"  
7   ]  
8 }
```

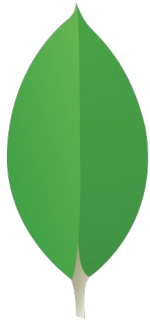
- Isso permite dados relacionados que podem ser agrupados naturalmente e na hora de buscar a ideia é simples (o mongo verifica o valor na lista BEM COMPLEXO):

```
1 colecao.find({  
2   "generos": "Drama"  
3 })
```



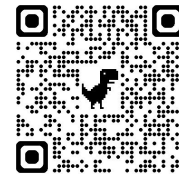

Quando usar MongoDB?

- Dados semi-estruturados
- Aplicações com mudanças frequentes
- Documentos complexos
- Sistemas distribuídos
 - Redes sociais, catálogos, logs, APIs JSON...



mongoDB®

carlos.melo@upe.br



Prof. Dr. Carlos Melo

<https://calendly.com/prof-casm>

 carlos.melo@upe.br